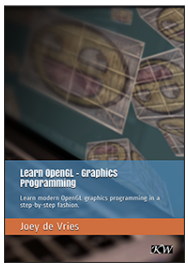
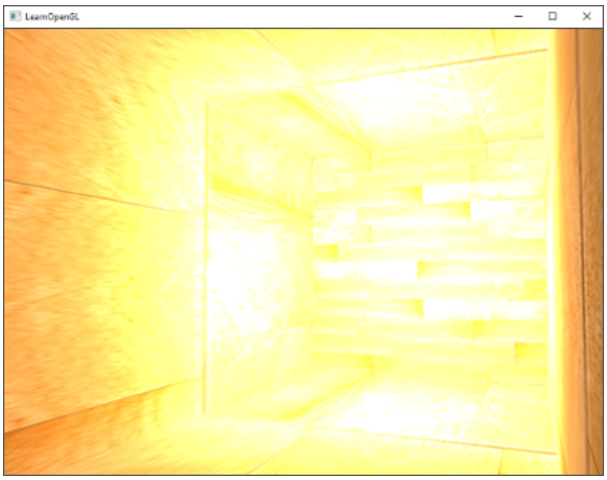
Introduction
Getting started
Lighting
Model Loading
Advanced OpenGL
Advanced Lighting
Advanced Lighting
Gamma Correction
Shadows
Normal Mapping
Parallax Mapping
HDR
Bloom
Deferred Shading
SSAO
PBR
In Practice
Guest Articles
Code repository
Translations
Privacy
About

PRINT EDITION



# HDR

Brightness and color values, by default, are clamped between `0.0` and `1.0` when stored into a framebuffer. This, at first seemingly innocent, statement caused us to always specify light and color values somewhere in this range, trying to make them fit into the scene. This works oké and gives decent results, but what happens if we walk in a really bright area with multiple bright light sources that as a total sum exceed `1.0`? The answer is that all fragments that have a brightness or color sum over `1.0` get clamped to `1.0`, which isn't pretty to look at:




Due to a large number of fragments' color values getting clamped to `1.0`, each of the bright fragments have the exact same white color value in large regions, losing a significant amount of detail and giving it a fake look.

A solution to this problem would be to reduce the strength of the light sources and ensure no area of fragments in your scene ends up brighter than `1.0`; this is not a good solution as this forces you to use unrealistic lighting parameters. A better approach is to allow color values to temporarily exceed `1.0` and transform them back to the original range of `0.0` and `1.0` as a final step, but without losing detail.

Monitors (non-HDR) are limited to display colors in the range of `0.0` and `1.0`, but there is no such limitation in lighting equations. By allowing fragment colors to exceed `1.0` we have a much higher range of color values available to work in known as high dynamic range (HDR). With high dynamic range, bright things can be really bright, dark things can be really dark, and details can be seen in both.

High dynamic range was originally only used for photography where a photographer takes multiple pictures of the same scene with varying exposure levels, capturing a large range of color values. Combining these forms a HDR image where a large range of details are visible based on the combined exposure levels, or a specific exposure it is viewed with. For instance, the following image (credits to Colin Smith) shows a lot of detail at brightly lit regions with a low exposure (look at the window), but these details are gone with a high exposure. However, a high exposure now reveals a great amount of detail at darker regions that weren't previously visible.



This is also very similar to how the human eye works and the basis of high dynamic range rendering. When there is little light, the human eye adapts itself so the darker parts become more visible and similarly for bright areas. It's like the human eye has an automatic exposure slider based on the scene's brightness.

High dynamic range rendering works a bit like that. We allow for a much larger range of color values to render to, collecting a large range of dark and bright details of a scene, and at the end we transform all the HDR values back to the low dynamic range (LDR) of [`0.0`, `1.0`]. This process of converting HDR values to LDR values is called tone mapping and a large collection of tone mapping algorithms exist that aim to preserve most HDR details during the conversion process. These tone mapping algorithms often involve an exposure parameter that selectively favors dark or bright regions.

When it comes to real-time rendering, high dynamic range allows us to not only exceed the LDR range of [`0.0`, `1.0`] and preserve more detail, but also gives us the ability to specify a light source's intensity by their *real* intensities. For instance, the sun has a much higher intensity than something like a flashlight so why not

No burnout
Pay maintain
does! Join th
Pled
opensourc
If you're running AdB
this site if you'd like
a lot); and no worrie

configure the sun as such (e.g. a diffuse brightness of `100.0`). This allows us to more properly configure a scene's lighting with more realistic lighting parameters, something that wouldn't be possible with LDR rendering as they'd then directly get clamped to `1.0`.

As (non-HDR) monitors only display colors in the range between `0.0` and `1.0` we do need to transform the currently high dynamic range of color values back to the monitor's range. Simply re-transforming the colors back with a simple average wouldn't do us much good as brighter areas then become a lot more dominant. What we can do, is use different equations and/or curves to transform the HDR values back to LDR that give us complete control over the scene's brightness. This is the process earlier denoted as tone mapping and the final step of HDR rendering.

## Floating point framebuffers

To implement high dynamic range rendering we need some way to prevent color values getting clamped after each fragment shader run. When framebuffers use a normalized fixed-point color format (like `GL_RGB`) as their color buffer's internal format, OpenGL automatically clamps the values between `0.0` and `1.0` before storing them in the framebuffer. This operation holds for most types of framebuffer formats, except for floating point formats.

When the internal format of a framebuffer's color buffer is specified as `GL_RGB16F`, `GL_RGBA16F`, `GL_RGB32F`, or `GL_RGBA32F` the framebuffer is known as a floating point framebuffer that can store floating point values outside the default range of `0.0` and `1.0`. This is perfect for rendering in high dynamic range!

To create a floating point framebuffer the only thing we need to change is its color buffer's internal format parameter:

```
glBindTexture(GL_TEXTURE_2D, colorBuffer);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA16F, SCR_WIDTH, SCR_HEIGHT, 0, GL_RGBA, GL_FLOAT, N
```

The default framebuffer of OpenGL (by default) only takes up 8 bits per color component. With a floating point framebuffer with 32 bits per color component (when using `GL_RGB32F` or `GL_RGBA32F`) we're using 4 times more memory for storing color values. As 32 bits isn't really necessary (unless you need a high level of precision) using `GL_RGBA16F` will suffice.

With a floating point color buffer attached to a framebuffer we can now render the scene into this framebuffer knowing color values won't get clamped between `0.0` and `1.0`. In this chapter's example demo we first render a lit scene into the floating point framebuffer and then display the framebuffer's color buffer on a screen-filled quad; it'll look a bit like this:

```
glBindFramebuffer(GL_FRAMEBUFFER, hdrFBO);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    // [...] render (lit) scene
glBindFramebuffer(GL_FRAMEBUFFER, 0);

// now render hdr color buffer to 2D screen-filling quad with tone mapping shader
hdrShader.use();
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, hdrColorBufferTexture);
RenderQuad();
```

Here a scene's color values are filled into a floating point color buffer which can contain any arbitrary color value, possibly exceeding `1.0`. For this chapter, a simple demo scene was created with a large stretched cube acting as a tunnel with four point lights, one being extremely bright positioned at the tunnel's end:

```
std::vector<glm::vec3> lightColors;
lightColors.push_back(glm::vec3(200.0f, 200.0f, 200.0f));
lightColors.push_back(glm::vec3(0.1f, 0.0f, 0.0f));
lightColors.push_back(glm::vec3(0.0f, 0.0f, 0.2f));
lightColors.push_back(glm::vec3(0.0f, 0.1f, 0.0f));
```

Rendering to a floating point framebuffer is exactly the same as we would normally render into a framebuffer. What is new is `hdrShader`'s fragment shader that renders the final 2D quad with the floating point color buffer texture attached. Let's first define a simple pass-through fragment shader:

```
#version 330 core
out vec4 FragColor;

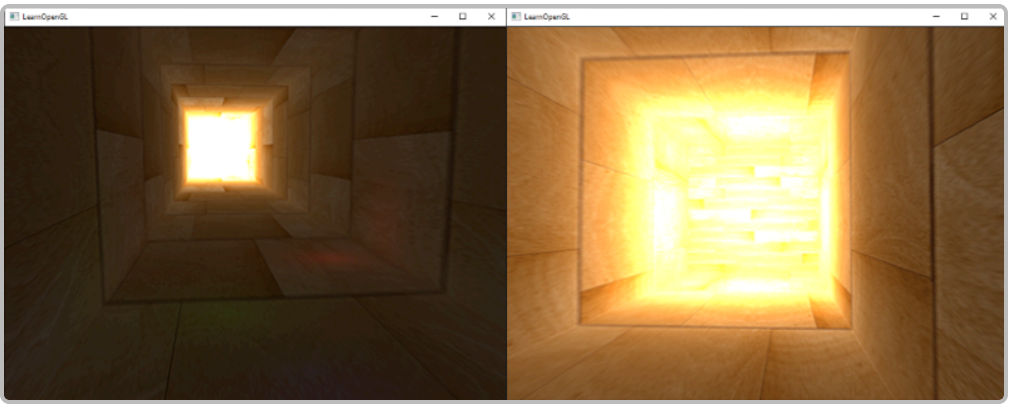
in vec2 TexCoords;

uniform sampler2D hdrBuffer;

void main()
{
```

```
    vec3 hdrColor = texture(hdrBuffer, TexCoords).rgb;
    FragColor = vec4(hdrColor, 1.0);
}
```

Here we directly sample the floating point color buffer and use its color value as the fragment shader's output. However, as the 2D quad's output is directly rendered into the default framebuffer, all the fragment shader's output values will still end up clamped between `0.0` and `1.0` even though we have several values in the floating point color texture exceeding `1.0`.



It becomes clear the intense light values at the end of the tunnel are clamped to `1.0` as a large portion of it is completely white, effectively losing all lighting details in the process. As we directly write HDR values to an LDR output buffer it is as if we have no HDR enabled in the first place. What we need to do is transform all the floating point color values into the `0.0` - `1.0` range without losing any of its details. We need to apply a process called tone mapping.

## Tone mapping

Tone mapping is the process of transforming floating point color values to the expected [`0.0`, `1.0`] range known as low dynamic range without losing too much detail, often accompanied with a specific stylistic color balance.

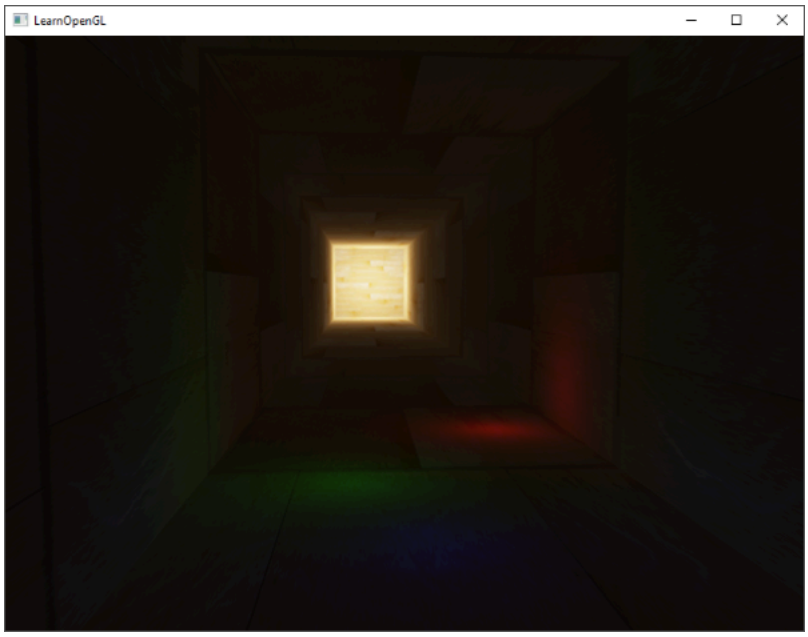
One of the more simple tone mapping algorithms is Reinhard tone mapping that involves dividing the entire HDR color values to LDR color values. The Reinhard tone mapping algorithm evenly balances out all brightness values onto LDR. We include Reinhard tone mapping into the previous fragment shader and also add a gamma correction filter for good measure (including the use of sRGB textures):

```
void main()
{
    const float gamma = 2.2;
    vec3 hdrColor = texture(hdrBuffer, TexCoords).rgb;

    // reinhard tone mapping
    vec3 mapped = hdrColor / (hdrColor + vec3(1.0));
    // gamma correction
    mapped = pow(mapped, vec3(1.0 / gamma));

    FragColor = vec4(mapped, 1.0);
}
```

With Reinhard tone mapping applied we no longer lose any detail at the bright areas of our scene. It does tend to slightly favor brighter areas, making darker regions seem less detailed and distinct:




Here you can again see details at the end of the tunnel as the wood texture pattern becomes visible again. With this relatively simple tone mapping algorithm we can properly see the entire range of HDR values stored in the floating point framebuffer, giving us precise control over the scene's lighting without losing details.

> Note that we could also directly tone map at the end of our lighting shader, not needing any floating point framebuffer at all! However, as scenes get more complex you'll frequently find the need to store intermediate HDR results as floating point buffers so this is a good exercise.

Another interesting use of tone mapping is to allow the use of an exposure parameter. You probably remember from the introduction that HDR images contain a lot of details visible at different exposure levels. If we have a scene that features a day and night cycle it makes sense to use a lower exposure at daylight and a higher exposure at night time, similar to how the human eye adapts. With such an exposure parameter it allows us to configure lighting parameters that work both at day and night under different lighting conditions as we only have to change the exposure parameter.

A relatively simple exposure tone mapping algorithm looks as follows:

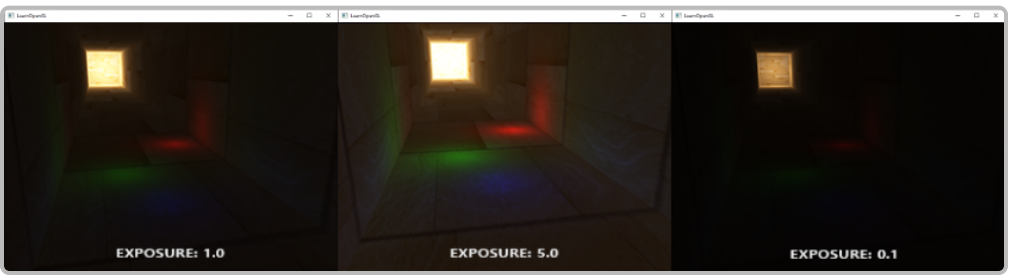
```
uniform float exposure;

void main()
{
    const float gamma = 2.2;
    vec3 hdrColor = texture(hdrBuffer, TexCoords).rgb;

    // exposure tone mapping
    vec3 mapped = vec3(1.0) - exp(-hdrColor * exposure);
    // gamma correction
    mapped = pow(mapped, vec3(1.0 / gamma));

    FragColor = vec4(mapped, 1.0);
}
```

Here we defined an `exposure` uniform that defaults at `1.0` and allows us to more precisely specify whether we'd like to focus more on dark or bright regions of the HDR color values. For instance, with high exposure values the darker areas of the tunnel show significantly more detail. In contrast, a low exposure largely removes the dark region details, but allows us to see more detail in the bright areas of a scene. Take a look at the image below to see the tunnel at multiple exposure levels:



This image clearly shows the benefit of high dynamic range rendering. By changing the exposure level we get to see a lot of details of our scene, that would've been otherwise lost with low dynamic range rendering. Take the end of the tunnel for example. With a normal exposure the wood structure is barely visible, but with a low exposure the detailed wooden patterns are clearly visible. The same holds for the wooden patterns close by that are more visible with a high exposure.

You can find the source code of the demo here.

### More HDR

The two tone mapping algorithms shown are only a few of a large collection of (more advanced) tone mapping algorithms of which each has their own strengths and weaknesses. Some tone mapping algorithms favor certain colors/intensities above others and some algorithms display both the low and high exposure colors at the same time to create more colorful and detailed images. There is also a collection of techniques known as automatic exposure adjustment or eye adaptation techniques that determine the brightness of the scene in the previous frame and (slowly) adapt the exposure parameter such that the scene gets brighter in dark areas or darker in bright areas mimicking the human eye.

The real benefit of HDR rendering really shows itself in large and complex scenes with heavy lighting algorithms. As it is difficult to create such a complex demo scene for teaching purposes while keeping it accessible, the chapter's demo scene is small and lacks detail. While relatively simple it does show some of the benefits of HDR rendering: no details are lost in high and dark regions as they can be restored with tone mapping, the addition of multiple lights doesn't cause clamped regions, and light values can be specified by real brightness values not being limited by LDR values. Furthermore, HDR rendering also makes several other interesting effects more feasible and realistic; one of these effects is bloom that we'll discuss in the next next chapter.

## Additional resources

- Does HDR rendering have any benefits if bloom won't be applied?: a stackexchange question that features a great lengthy answer describing some of the benefits of HDR rendering.
- What is tone mapping? How does it relate to HDR?: another interesting answer with great reference images to explain tone mapping.

57 Comments

Login ▾

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS

Name

10 Share

Best Newest Oldest

**Sein Lee**
6 years ago

I got it
View – uploads.disquscdn.com
View – uploads.disquscdn.com
View – uploads.disquscdn.com
View – uploads.disquscdn.com

12 0 Reply

**Andrei Despinoiu**
4 years ago edited

I wrote a working implementation for automatic exposure (aka "eye adaptation"), based on one of Joey's comments from below (thanks, man!):

View – uploads.disquscdn.com

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, screenTexture);

if (isEyeAdaptationEnabled)
{
        glGenerateMipmap(GL_TEXTURE_2D); // Generate mipmaps every frame
        glm::vec3 luminescence;
        glGetTexImage(GL_TEXTURE_2D, 10, GL_RGB, GL_FLOAT, &luminescence); // Read the value from
        const float lum = 0.2126f * luminescence.r + 0.7152f * luminescence.g + 0.0722f * luminesc

        const float adjSpeed = 0.05f;
        sceneExposure = glm::lerp(sceneExposure, 0.5f / lum * sceneExposureMultiplier, adjSpeed);
        sceneExposure = glm::clamp(sceneExposure, sceneExposureRangeMin, sceneExposureRangeMax); /

        pScreenShader->SetFloat("uExposure", sceneExposure);
}
```

This needs to run every frame shortly before drawing the screen quad. Note that "glGenerateMipmap()" is kind of expensive... If I disable VSync (so that it doesn't limit my application

see more

10 0 Reply

**Samuelintheforest**
4 years ago edited

Thank you! My result:
It has some issues but (like the exposure not applied in the reflection and cubemap, and I still messing with getting a good shadow map that is not that pixelated at the edge) but yeah good stuff 😁. Actually I used **ruby** as a language
My 3d engine so far – disq.us

5 0 Reply

**Alexander Christensen**
4 years ago

I found an interesting source of information, and comparison, of different HDR techniques: https://www.slideshare.net/...

Notably, this presentation explicitly mentions the Reinhard algorithm as yielding poor results compared to what they call "filmic tonemapping", adopted after Kodak film techniques. Any thoughts on this? The presentation is from 2012, but still seems quite relevant..

1 0 Reply

**Mahdi Daneshvar**
4 years ago edited

Here is what I got: Gamma correct HDR lighting. I am too happy with the results.

The level is made in Radiant. Its actually in DirectX but the shader stuff are quite easy to convert from GLSL to HLSL.
View – uploads.disquscdn.com

1 0 Reply



**Sarah Qahtan**
7 years ago

Hello everyone please any one can tell me whats wrong here .. after I run HDR code I got this View – uploads.disquscdn.com

1 0 Reply



**Ano Nym** → Sarah Qahtan
5 years ago

did you fix that back then? Now, I have the same problem :/ and it seems that there is nothing wrong with the path to the texture png

0 0 Reply



**Andrei Despinoiu** → Ano Nym
4 years ago edited

Set the exposure uniform to start off at 1.0:

```
uniform float uExposure = 1.0; // Start off at 1.0
```

Otherwise you have to send it as "1.0" at least once before you start rendering, because if the exposure uniform is zero (which it is, by default, if you don't specify anything), the output will be zero, meaning black.

And since the final output is a vec4, make sure the last parameter (the alpha) is 1.0, otherwise if it's 0.0 you will see whatever color glClear() was set to, which is what is displayed behind the screen quad (default is black, but if you follow these tutorials it might be blue, while the previous lessons had it as white). So that means:

```
oFragColor = vec4(mapped, 1.0); // 1.0 instead of 0.0
```

1 0 Reply



**MadPotatoe** → Sarah Qahtan
5 years ago edited

Yes same here but no answer :C

EDIT: got it its because at the Terminal it says that the texture wasnt loaded try looking at that :D

0 0 Reply



**LooseIdiot**
9 years ago

May I know where did you get the exposure tone mapping algorithm?

1 0 Reply



**JoeyDeVries** Sensei → LooseIdiot
9 years ago

I have no idea anymore, sorry. Probably got it somewhere from the internet when I first started playing with HDR.

0 0 Reply



**redhotchilialex**
9 years ago edited

Hey I have a full screen quad which i attached an hdr image to, then i rendered to a framebuffer and tried your shaders to do tone mapping. The result is a white screen. Funny thing is if I insert in the tone mapping fragment shader something like the pseudocode down there:

if 0 < result < 1 set result = vec3(0,0,1);

it actually returns a blue screen. So I'm sure the resulting color is not white, but still, white screen!

1 0 Reply



**Ano Nym** → redhotchilialex
5 years ago

do you remember how you fixed it 5 years ago? I also have a blue screen and I have no clue

do you remember how you fixed it 3 years ago? I also have a blue screen and I have no clue what the problem could be. Help would be much appreciated!

0 0 Reply



**redhotchilialex** Ano Nym

5 years ago

My problem was not that I got a blue screen but that the image was white with the proposed shader. The solution was just playing around with the exposure parameter.

0 0 Reply



**MrBubbles**

10 years ago

Can the average brightness of the image be calculated like you did in the framebuffers tutorial:

color = texture(screenTexture, TexCoords);
float average = 0.2126 * color.r + 0.7152 * color.g + 0.0722 * color.b;
color = vec4(average, average, average, 1.0);

Or this averages the color for a single fragment?
I'm asking about the eye adaption effect

1 0 Reply



**JoeyDeVries** Sensei MrBubbles

10 years ago edited

Exactly! The idea of getting the average brightness of the scene is to calculate this for each fragment and then take the average of **all** the fragment's brightness (or luminescence) values to get the scene's average brightness. So your code only does this for a single fragment but you want to calculate this for ALL fragments.

The first idea that probably comes to mind is to iterate over all of the scene's pixels (by reading from a texture or using `glReadPixels` to read the content of a framebuffer) and calculate the brightness for each fragment and average them at the end. While this does work it is **extremely** expensive to do as you'd have to do this for A LOT of pixels and also on the CPU that unlike the GPU is not optimized for stuff like this.

What people thus generally do is to take two framebuffer objects and *pingpong* between two textures where each iteration renders a smaller version of the texture to the other framebuffer. In this case we'd take the image of the scene and render it to a framebuffer twice as small, then we switch it around and render the resulting texture (again now twice as small) to the original framebuffer and so on. By using linear filtering OpenGL automatically takes the average of of the nearby fragments of each coordinate when downscaling so if we continue this operation until the texture size is equal to 1x1 (or 16x16 and then iterate over all the pixels) we get a single pixel that represents the average

see more

4 0 Reply



**Nielsbishere** JoeyDeVries

7 years ago

I know this has been posted 3 years ago, but since I was reading through this tutorial right now, I thought it would be handy to talk about better ways of doing this.
Since OpenGL 4.3, compute shaders have been introduced and these make it so you don't have to move data from the GPU to the CPU to do calculations like these. Something you could do is combine bloom with this exposure calculation in modern pipelines.

This would work as following:
Divide your screen up in tiles (32x32 or 16x16, depends on hardware and per group calculations).
Dispatch compute with groups [ ceil(x / tileSize), ceil(y / tileSize), 2] and threads [ tileSize, tileSize, 1 ].
Since every group has to follow the same branch, and every group has the same z value, you can assume that branches don't have a heavy overhead (when comparing z).
z = 0:
fetch the bloom result and add it to the pixel
calculate the lumen of a pixel and map it as a ushort (it won't be 100% accurate, but good enough for calculating the rough brightness of the scene).
use groupshared atomics to add every ushort to a uint in a buffer
average this by the thread count
add this ushort to a uint in a shader buffer

z = 1:
per pixel, convert this value from uint to float and normalize it (val / ceil(x / tileSize) / ceil(y / tileSize)).
use that brightness value to tonemap color + bloom result.

2 0 Reply



**Nielsbishere** → JoeyDeVries
7 years ago

I know this is

0 0 Reply



**MrBubbles** → JoeyDeVries
10 years ago

Interesting, I though that rendering a lot of times to a framebuffer will kill performance, but now that I see this I am going to try it!
Another thing that I am thinking is to generate mipmaps for the framebuffer and read the last level in the fragment shader, but I dont know if it is possible or not.
Thank you for such a complete answer, and thank you for your tutorial!
I'll let you know if I make it! :D

0 0 Reply



**JoeyDeVries** Sensei → MrBubbles
10 years ago edited

Yeah it definitely costs some performance, but it beats having to calculate the luminence of each pixel on a 1920x1080 resolution monitor ;)

I haven't implemented it myself, but I did read a discussion some time ago about calculating a scene's brightness with mipmaps and if I recall correctly it seemed a viable option. It seems easier to implement and probably faster too so I'd definitely give that a shot too :)

0 0 Reply



**Noman Ahmed Khan**
7 months ago

View – uploads.disquscdn.com

0 0 Reply



**Octavius Ace**
4 years ago edited

Double-posted - pleas ignore
Correct version in PBR specular page comments

0 0 Reply



**Vaani-del**
4 years ago edited

Hey Joey, I'd love to see you talk about different tonemapping curves such as ACES (though I'd rather you discuss a *proper* ACES curve, one that properly follows ACES spec and has adjustable slope, toe, shoulder, etc) and Gran Turismo

0 0 Reply



**Andrei Despinoiu**
4 years ago edited

View – uploads.disquscdn.com

I'm surprised how well it changes the mood. It opens up possibilities to create desert scenes, night scenes, horror scenes, even nuclear explosions. Try this: Make a spotlight really really bright (e.g. 1500), then set the exposure to something really low, like 0.01. Boom, instant horror. :P

Thank you for writing this tutorial.

Any reason why "GL_RGBA16F" is used instead of "GL_RGB16F"? I mean, why would an alpha channel be needed for the color buffer? Seems like wasted memory, unless you plan on blending it with something in the background.

The 'gamma' variable in the fragment shader isn't needed if you have `glEnable(GL_FRAMEBUFFER_SRGB);` before creating the color buffer and load your textures using "GL_SRGB" for textures with 3 channels and "GL_SRGB_ALPHA" for textures with 4 channels. OpenGL will gamma correct it automatically at

and "GL_SRGB_ALPHA" for textures with 4 channels. OpenGL will gamma correct it automatically at the end.

You can still keep that variable, maybe as a uniform, just set it to 1.0 by default, not 2.2. Otherwise you get a milky white scene, because you would be gamma correcting a fragment that is gonna get gamma corrected at the end anyway, resulting in a more bright washed out image. Then you can decrease that uniform for an even more horror look.

0 0 Reply



**Ano Nym**
5 years ago

View – uploads.disquscdn.com

I've implemented the program like above but I still got nothing than the followng output: (I already checked the path to the file. there is everything okay regarding this). Does anyone know what the problem could be?

0 0 Reply



**iaomw**
6 years ago

Should I do tone mapping to all the pixels? What if I do it only for pixels derived from HDR texture?

0 0 Reply



**Campus**
7 years ago

Awesome tutorial as always.
I have noticed that with HDR my colors tend to look more "bland", lower in saturation. Is there a way to correct for that?
Let me point out that I am not applying gamma correction in the shader, since I am already gamma-correcting with glEnable(GL_FRAMEBUFFER_SRGB).
Could that be the reason?

View – uploads.disquscdn.com

0 0 Reply



**Donlod** → Campus
4 years ago

if you apply gamma correction in your shader you have to also set the textures internal format to sRGB.

If instead of doing the correction yourself you use glEnable(GL_FRAMEBUFFER_SRGB) you still have to set the internal format to srgb. the output is nearly the same as doing it yourself in the shader. 2.2 is just an approximation.

0 0 Reply



**Thomas** → Campus
7 years ago edited

I don't know if you've fixed this but you can do a colour correction algorithm or simply increase the light levels. HDR can handle higher lighting levels and I found that after changing to HDR and increasing lighting levels it look much more realistic. Also, there is lower saturation due to how HDR is more realistic. Notice how the bright wood without HDR becomes a yucky yellow while HDR slowly becomes a white which is a lot more realistic. *Lighting levels is the intensity of your lights.

0 0 Reply



**PersonOfInterest**
7 years ago

"For instance, the image below shows a lot of detail at brightly lit
regions with a high exposure (look at the window), but these details are
gone with a low exposure."

I think you got high and low backwards here.

0 0 Reply



**JoeyDeVries** Sensei → PersonOfInterest
7 years ago

You're right; fixed it, thanks!

0 0 Reply

**Jason Cai**
8 years ago edited

Hi Joey, it's me, again. Thanks to your tutorials.
Things might go wrong when I try your code. I didn't get the image shown in your tutorial. Here are my images.
View – uploads.disquscdn.com
The left image is rendered without any tone mapping algorithm. And the right image is rendered by reinhard method. But it is far away from your image in 'Floating point framebuffers' section. I couldn't tell why, could you give me an answer?

0 0 Reply

**JoeyDeVries** Sensei → Jason Cai
8 years ago

Not sure, it's been a while since I've set up this demo. Looking at the source code the wood texture image is loaded with SRGB sample mode on the HDR version; maybe in the non-HDR version it wasn't loaded with SRGB flags (so loaded as normal) or vica versa? This could darken/brighten the texture image based on which mode you're loading/configuring the texture.

0 0 Reply

**Jason Cai** → JoeyDeVries
8 years ago

Well, you corrected me a mistake. I really didn't make the texture image to SRGB sample mode. Unfortunately, when I was expecting the problem was resolved, I found that it's not the reason.Then, I remembered that the image seems a bit like the situation in Gamma correction tutorial. I tried with attenuation in linear distance. And the result image is similar to yours.
View – uploads.disquscdn.com
Could it be this reason? Looking forward to a response!

0 0 Reply

**JoeyDeVries** Sensei → Jason Cai
8 years ago edited

Ahh yes, that could very well be; the attenuation function you're using should be based on whether you do your lighting calculations in gamma or linear. If we're gamma-correcting (and thus doing our lighting in linear space) you want to use something like the inverse square falloff function as found in the lighting shader:

```
// attenuation (use quadratic as we have gamma correction)
float distance = length(fs_in.FragPos - lights[i].Position);
result *= 1.0 / (distance * distance);
```

0 0 Reply

**Jason Cai** → JoeyDeVries
8 years ago edited

That's it! If so, your image in "Floating point framebuffers" section might use something like this:
float distance = length(fs_in.FragPos - lights[i].Position);
result *= 1.0 / distance; // attenuation linear
And in next section, the attenuation code become:
float distance = length(fs_in.FragPos - lights[i].Position);
result *= 1.0 / (distance * distance); // attenuation quadratic
So, the image in last section is so bright and next become so dark, which confused me a lot. Because the difference is not as obvious as your action between directly transform HDR values to LDR values and use Reinhard tone mapping, is it?

0 0 Reply

**Wanter**
9 years ago

Hello. If we are about to use underexposed image and overexposed image for getting hdr image, how would we merge those 2 images to get final values?

0 0 Reply

**Huxa** → Wanter
9 years ago

Also, the best openGL tutorial by far. Thanks for doing it!

0 0 Reply



**jkster**

9 years ago

Could you add a section about HDR 10bit to this tutorial?
For the new HDR TVs and upcoming HDR monitors.

0 0 Reply



**Celal**

9 years ago

Hi, thank you for all your tutorials those are best in net. I have one question, I scale my tunnel however I cannot scale also hr its stay middle. How can I scale also HR shine ?

0 0 Reply



**卡卡西**

9 years ago

I'm wondering, why not directly using tone mapping in the end without using any hdr float framebuffer texture :)

0 0 Reply



**JoeyDeVries** Sensei → 卡卡西

9 years ago

You can definitely do that and in this case it would make the most sense; but I've kept it in there as in more complex scenes this becomes a more general use case (as well as to teach the importance of keeping intermediate HDR results in floating point framebuffers); I made a note box at the tone mapping section to highlight htis; thank you! :)

1 0 Reply



**卡卡西** → JoeyDeVries

9 years ago

thanks very much for your answer, helps a lot.

1 0 Reply



**Bowen**

10 years ago

I added the code in this tutorial to my own renderer, and it made everything brighter and more grey. The colors seem less vibrant than before. Is that supposed to happen?

0 0 Reply



**JoeyDeVries** Sensei → Bowen

10 years ago

I also included Gamma Correction code into the HDR shader, which makes your scene brighter if you haven't applied this before. Check out the Gamma Correction tutorial if you want to know more, or leave the gamma correction line out of the shaders.
As for everything else, applying tone correction to your final scene will most likely change the general feel/colors of your scene a bit. Try to tweak and play with the parameters to get the effect you want.

1 0 Reply



**Christian**

10 years ago

From where can I download the lighting vertex and fragment shader used to render the scene?

// Setup and compile our shaders
Shader shader("lighting.vs", "lighting.frag");

0 0 Reply



**JoeyDeVries** Sensei → Christian

10 years ago

It's basically just some basic Phong lighting shaders (not really relevant for this tutorial), but if you want you can find the full source code (including all shaders) on Github here.

0 0 Reply



**Christian** → JoeyDeVries

10 years ago

Thanks

Thanks

0 0 Reply



**Sebastian**

10 years ago

I use NVIDIA card and hdr not working, only black screen - no error

0 0 Reply



**JoeyDeVries** Sensei → Sebastian

10 years ago

Check if your framebuffers are complete, make sure the colorbuffer's texture formats are set correctly and check each individual colorbuffer texture (by rendering it to the 2D plane quad, simply bind a different texture to check its results). Try to pinpoint where things go wrong so I can better help you.

0 0 Reply

Load more comments

Subscribe Privacy Do Not Sell My Data